Science of Computer Programming 247 (2026) 103357



Contents lists available at ScienceDirect

Science of Computer Programming

journal homepage: www.elsevier.com/locate/scico





Detecting duplicate vulnerability records across databases

Kangliang Zhu [a], Wenhua Yang [a,b,*], Minxue Pan [b], Yu Zhou [a]

[a] College of Computer Science and Technology, Nanjing University of Aeronautics and Astronautics, Nanjing, China
[b] State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing, China

ARTICLE INFO

Keywords:
Duplicate vulnerability
Cosine similarity
BERT
Fine-tuning
Dataset

ABSTRACT

Vulnerability databases are critical repositories that aggregate information about known security vulnerabilities across various software products. However, the existence of multiple, heterogeneous databases often leads to duplicate vulnerability records, necessitating significant manual effort by maintainers to identify and consolidate these duplicates. This study addresses the challenge of detecting duplicate vulnerabilities across different databases by proposing a combined method that integrates cosine similarity measures with a fine-tuned BERT-based language model. We constructed a comprehensive duplicate vulnerability dataset by analyzing records from prominent databases such as CVE, OSV, and the GitHub Advisory Database. Our method was evaluated against several baseline techniques, including similarity-based and deep learning-based approaches, demonstrating superior performance across multiple metrics, including Hit Rate@N, Mean Reciprocal Rank (MRR), Mean Rank, and Median Rank. Additionally, our method proved effective in practical scenarios involving ongoing database maintenance, showcasing its ability to generalize to unseen data. The findings highlight the potential of integrating traditional similarity measures with advanced language models to enhance the accuracy and efficiency of duplicate vulnerability detection, thereby facilitating more reliable vulnerability management.

1. Introduction

Vulnerability databases are centralized repositories containing detailed information about known security vulnerabilities in various software products. These databases play a vital role in helping organizations, developers, and security professionals maintain more secure software environments and respond effectively to potential threats [1]. However, different organizations and initiatives have created their own databases to address specific needs or gaps in existing resources, leading to separate, established databases. For example, public vulnerability databases include the Common Vulnerabilities and Exposures (CVE), the National Vulnerability Database (NVD), ExploitDB, and SecurityTracker [2]. Additionally, there are open-source vulnerability databases such as the GitHub Advisory Database and the Open Source Vulnerabilities (OSV) database. These vulnerability databases aggregate data from various sources, including user-submitted reports and information from online forums. They often reference data from one another as well. For instance, many databases crawl and incorporate CVE entries. When a new vulnerability appears in the CVE, they add it to their own database or map the new CVE to an existing vulnerability in their records. However, because the CVE assignment process can be time-consuming, new vulnerabilities may already exist in other databases before they receive a CVE ID, and they may not yet be mapped to the corresponding CVE entries. To our knowledge, no existing studies in the literature have explicitly addressed this

* Corresponding author at: College of Computer Science and Technology, Nanjing University of Aeronautics and Astronautics, Nanjing, China.
E-mail addresses: willzhu@nuaa.edu.cn (K. Zhu), ywh@nuaa.edu.cn (W. Yang), mxp@nju.edu.cn (M. Pan), zhouyu@nuaa.edu.cn (Y. Zhou).

https://doi.org/10.1016/j.scico.2025.103357
Received 31 December 2024; Received in revised form 23 April 2025; Accepted 24 June 2025

Available online 7 July 2025
0167-6423/© 2025 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

issue through the application of automated methodologies. Currently, mapping these vulnerabilities often requires manual effort from database maintainers. For instance, the NVD's official documentation [3] outlines the process of reviewing and mapping vulnerabilities to CVE entries, shedding light on the manual procedures involved in updating and maintaining the database. Similarly, the official documentation [4] for OSV notes that while automated tools like OSV-Scanner can help identify and document vulnerabilities, human intervention is still needed to validate and link vulnerability information. Furthermore, discussions and shared experiences within the security community often underscore the need for manual involvement in associating vulnerability records with CVE entries [5].

Moreover, not all vulnerabilities are assigned a CVE ID. Some vulnerabilities, particularly those not reported through official channels or deemed less critical, may not receive a CVE assignment but still exist in various vulnerability databases. When databases reference each other's data, they need to confirm whether the referenced vulnerabilities already exist in their own collections, which also relies on manual processing by the maintainers. Therefore, given the existence of multiple vulnerability databases and the interactions among them, the challenge of identifying duplicate vulnerabilities across different databases becomes significant. Addressing this problem not only helps databases maintain more accurate and comprehensive data but also enhances the efficiency of vulnerability management and facilitates better coordination among security professionals.

There are several reasons why the same vulnerability may appear in different databases. Multiple developers might independently report the same vulnerability to different databases, and they may struggle to identify the true root cause when reporting it. For instance, a developer could discover a vulnerability while using a package and report it to that package's GitHub repository, not realizing that the actual issue originates from a dependent package. Other developers might report the same vulnerability on the dependent package's page or on the pages of other packages that use it. In December 2024, a security advisory[1] was issued for the `rails-html-sanitizer` gem, highlighting a potential Cross-Site Scripting vulnerability. This issue occurred because certain configurations of `rails-html-sanitizer` allowed an attacker to inject content when HTML5 sanitization was enabled, and specific tags were permitted. Further investigation revealed that the root cause of the vulnerability was tied to the `nokogiri` gem, a dependency of `rails-html-sanitizer`. The same issue was also reported in the GitHub repository for nokogiri.[2] Such scenarios can lead to multiple records of the same vulnerability across different databases. Additionally, reports from different developers about the same vulnerability may contribute to the duplication of vulnerabilities within a database. For example, the GitHub Advisory Database, which aggregates vulnerabilities in open-source projects, may inadvertently contain duplicate entries due to overlapping reports. One example is a vulnerability[3] reported in the GitHub Advisory Database, describing a path traversal issue in `Reposilite` when handling extended javadoc files, which could allow arbitrary file reading. The same vulnerability[4] was reported in another advisory entry within the same database. After review by maintainers, it was confirmed that these entries were duplicates. The identification and recognition of many duplicate vulnerabilities often rely on manual effort. While it might seem intuitive that the same vulnerability would share similarities that aid in identification, our pilot study (Section 2) revealed that these similarities are not as significant as expected. Moreover, considerable differences exist in the fields used by various vulnerability databases; some focus more on code (such as ExploitDB), while others emphasize textual descriptions. Therefore, identifying duplicate vulnerabilities across different databases requires extra effort.

Existing research on vulnerabilities is extensive, with many studies focusing on vulnerability detection using program analysis methods [6] and machine learning techniques [7–11]. One category of research concentrates on detecting recurring vulnerabilities—vulnerabilities stemming from the same root cause occurring in different software or different language versions of the same software [12–14]. These studies aim to identify vulnerabilities that share a common origin but appear in different contexts. However, they do not address the identification of duplicate vulnerabilities across different databases, which is the focus of our work. Other related research [15–20] includes vulnerability repair, analysis of vulnerability impact scope, and predicting the severity of vulnerabilities. To the best of our knowledge, there is currently no dedicated research on identifying duplicate vulnerabilities across different vulnerability databases.

Similar to duplicate vulnerability identification, considerable research has been conducted on detecting duplicate bug reports [21–28]. These studies often employ natural language processing and information retrieval techniques to identify reports that describe the same underlying issue, enhancing bug triage and resolution efficiency. However, unlike vulnerabilities, bug reports tend to focus on the symptoms observed by users after a defect occurs, leading to higher similarity among duplicate reports. In contrast, vulnerability reports typically describe how a vulnerability is triggered, and duplicate vulnerability reports may not exhibit the same level of textual similarity. Additionally, duplicate bug report detection has been researched for over a decade, resulting in several well-constructed and publicly available datasets. In contrast, duplicate vulnerability detection has not been extensively studied, and there are currently no available, well-constructed datasets for this problem. The absence of such datasets hinders the advancement of research in duplicate vulnerability identification.

To address the lack of research on duplicate vulnerability identification and the absence of related datasets, we investigate these issues in this paper. First, we conducted an in-depth analysis of several common vulnerability databases, including CVE, OSV, and the GitHub Advisory Database. Utilizing identifiers such as CVE IDs and GHSA numbers from the GitHub Advisory Database, we extracted pairs of duplicate vulnerabilities that had been previously marked by database maintainers. We then constructed and publicly released a duplicate vulnerability dataset[5] to facilitate future research. The creation of this dataset provides a foundation for developing and

[1] https://github.com/rails/rails-html-sanitizer/security/advisories/GHSA-w8gc-x259-rc7x.
[2] https://github.com/sparklemotion/nokogiri/issues/3351.
[3] https://github.com/advisories/GHSA-82j3-hf72-7x93.
[4] https://github.com/advisories/GHSA-w7c4-5w4f-jm3g.
[5] https://zenodo.org/records/15234053.

evaluating methods for duplicate vulnerability identification, which can accelerate progress in this area. Additionally, to enhance the identification of duplicate vulnerabilities across different databases, we propose a method that combines similarity-based measures with a pre-trained model, leveraging the strengths of both approaches. We compare this method against several baseline techniques, some of which are derived from duplicate bug report detection research, providing further context and insights into our method's performance. Experimental evaluations confirm that our method consistently outperforms several baseline techniques, delivering superior accuracy while maintaining computational efficiency.

Overall, this paper makes the following key contributions:

- **Pilot Study Revealing Duplicate Vulnerabilities:** We conducted a pilot study to uncover the existence of duplicate vulnerabilities across different vulnerability databases. This study investigates the similarities of these duplicates in various databases, highlighting the challenges in identifying duplicate vulnerabilities due to differences in database structures and content.
- **Construction of a Public Duplicate Vulnerability Dataset:** Recognizing that the issue of duplicate vulnerabilities has not been extensively researched and that no publicly available datasets exist, we constructed a duplicate vulnerability dataset. By thoroughly analyzing various information from multiple vulnerability databases, we have created and released this dataset (footnote 5) to facilitate future research in this area.
- **Proposal and Empirical Evaluation of a Combined Identification Method:** We proposed a combined method for identifying duplicate vulnerabilities in different vulnerability databases. To determine the most effective approach for duplicate vulnerability identification, we conducted empirical comparisons with other related methods from different categories.

The remainder of this paper is organized as follows. In Section 2, we introduce our pilot study, which motivates the need for this research. Section 3 details the construction of our duplicate vulnerability dataset. In Section 4, we present our proposed method for identifying duplicate vulnerabilities across different vulnerability databases. Section 5 provides the evaluation and comparison of our method with other approaches. Section 6 discusses threats to validity. Section 7 discusses the limitations of our work and explores potential directions for future research. We review related work in Section 8. Finally, Section 9 concludes the paper and outlines directions for future research.

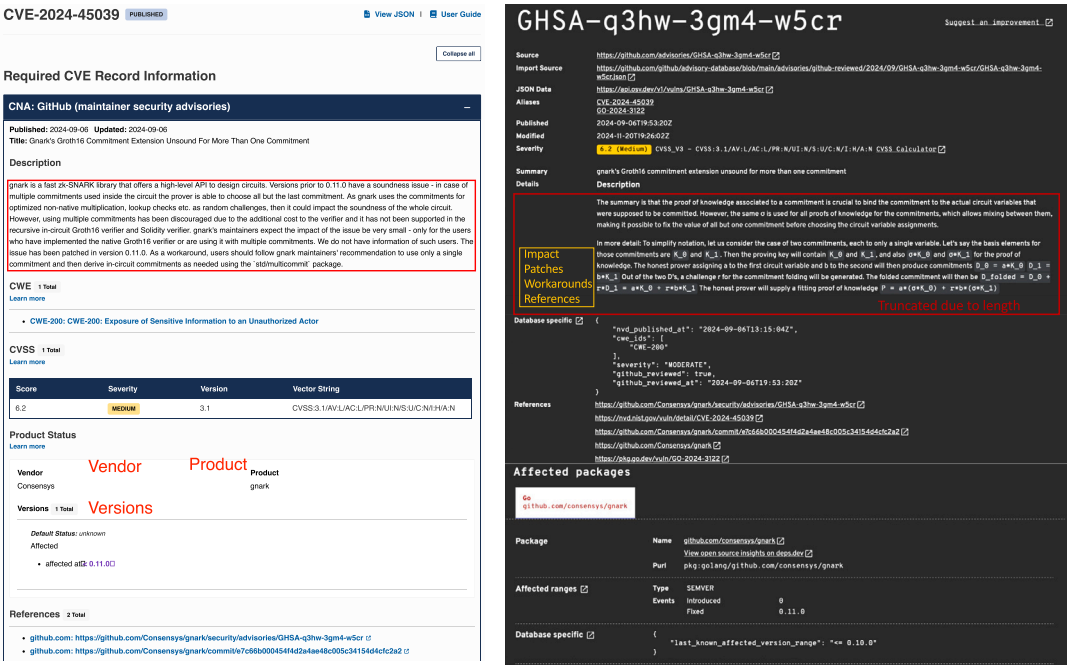
2. Motivation

Duplicate vulnerabilities exist across different vulnerability databases, which often reference each other. Developers typically consult multiple databases during vulnerability assessment and remediation. Therefore, identifying duplicates is crucial for effective vulnerability management and for reducing confusion among developers. As noted in Section 1, many duplicate identifications are often carried out manually after the same vulnerability has been independently recorded in multiple databases. For instance, due to the nature of the CVE assignment process, a vulnerability may appear in other databases before a CVE ID is issued. Once the CVE ID is available, corresponding entries in other databases need to be manually linked to ensure consistency. Our work seeks to automate this process by identifying duplicate vulnerabilities and assisting in the accurate mapping between them.

In this section, we present a pilot study to uncover such duplicates in practice and investigate how they are represented. We analyzed two widely used vulnerability databases: CVE and OSV. Our objective was to identify duplicate vulnerabilities and examine their representations. To accurately determine duplicates, we relied on the unique CVE IDs assigned to each vulnerability in the CVE database. The CVE system serves as a standard reference in the cybersecurity community, providing consistent and unique identifiers that enhance interoperability among databases and facilitate vulnerability management. We observed that maintainers of other databases often reference CVE IDs by comparing newly released CVEs with their own records. If they identify a matching vulnerability, they associate it with the corresponding CVE ID. For example, OSV uses the "Aliases" field for this purpose, and some databases even incorporate descriptions from CVE. These duplicate markings are added manually by professional database maintainers after thorough verification, making them reliable indicators of duplicate vulnerabilities. Given the extensive use and recognition of CVE IDs, we utilized them in our pilot study to identify duplicate vulnerabilities. Specifically, if two vulnerabilities from different databases share the same CVE ID, we consider them duplicates. We analyzed duplicate vulnerabilities between the CVE and OSV databases from 2021 to 2023.

Using the method described above, we identified 19,919 duplicate vulnerability pairs during this period. We examined the creation and publication dates of these entries and found that in 10,502 of the 19,919 pairs (52.7%), the CVE ID was published after the corresponding OSV record had been created. In such cases, the OSV entries were later associated to the CVE IDs through the "Aliases" field. This indicates that the association between the OSV record and the CVE ID must have been added after the OSV entry was initially created, as the CVE ID did not yet exist at that time. For the remaining pairs, the CVE ID was published before the OSV record was created. However, because detailed revision histories of OSV records are not publicly available (only the most recent modification timestamp is shown), we cannot determine the exact point at which the CVE ID was added. Still, our analysis reveals that over 97.9% of these OSV entries have been modified since their creation. Only around 2% appear to have included the CVE ID from the outset, likely because OSV referenced existing CVE information. For the rest, it is possible that the CVE ID was added after the record was first created. Overall, our findings suggest that for at least half of the duplicate vulnerability pairs between CVE and OSV, the duplication association between the entries was made after their initial creation. These cases require explicit efforts to identify and link the duplicate records across databases.

The identification of duplicate vulnerabilities depends on how they are represented across different vulnerability databases. To investigate this, we compared the representations of these vulnerability records and observed significant differences in their structure,



(a) CVE (b) OSV

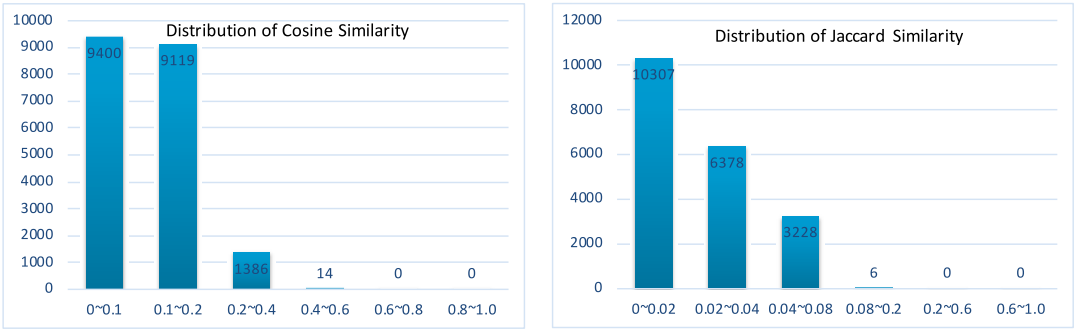
Fig. 1. Records of the same vulnerability in two different vulnerability databases.

content focus, and metadata presentation. Fig. 1 (a) and (b) present vulnerability reports for the same issue from the CVE and OSV databases, respectively. Although both reports describe the same underlying vulnerability, their field structures have some differences. For example, the CVE report[6] includes the associated CWE type, while the OSV report[7] features an "Aliases" field that is absent in the CVE version. Beyond structural differences, the reports also vary in the depth and technical detail of their descriptions. As shown in Fig. 1 (a), the CVE report provides a brief summary, mentioning the affected library, the cause of the issue, and its potential impact. In contrast, the OSV report offers a more detailed explanation, including code references and concrete examples illustrating how the vulnerability manifests. Due to space limitations, only part of the OSV description is shown in the figure, and the full version is available online (footnote 7). It also includes additional information such as the impact, available patches, and affected workloads. To quantify the textual differences between the two reports, we applied a basic natural language processing pipeline (e.g., tokenization, lowercasing, stopword removal) and computed their cosine similarity [29] and Jaccard similarity [30] scores. The resulting cosine similarity is 0.136, and the Jaccard similarity is close to 0, indicating minimal textual overlap between the two descriptions.

To further quantify the differences in how duplicate vulnerabilities are represented, we applied two widely used similarity metrics (i.e., cosine similarity and Jaccard similarity) to the 19,919 duplicate vulnerability pairs identified earlier. These metrics enable us to measure the textual similarities between duplicate vulnerability records. Cosine similarity assesses the overall similarity of the textual content by measuring the cosine of the angle between two non-zero vectors. Jaccard similarity focuses on the overlap of unique terms between descriptions, capturing shared vocabulary. We extracted the records for each pair of duplicate vulnerabilities from the two databases, applied the previously mentioned basic natural language processing techniques, and computed the similarity of the full records using the two metrics.

Fig. 2 presents the similarity distribution of 19,919 duplicate vulnerability pairs across different databases. We observe that, regardless of the similarity metric used, the similarity scores among these duplicate pairs are generally low. Specifically, for cosine similarity, 9,400 pairs have scores below 0.1, and another 9,119 pairs fall between 0.1 and 0.2, resulting in an average cosine similarity of only 0.115. The Jaccard similarity scores are even lower: 10,307 pairs have values below 0.02, and 6,378 pairs fall between 0.02 and 0.04. Almost all pairs have Jaccard scores below 0.08, with an overall average of just 0.018. In general, cosine similarity values approaching 1 indicate high similarity, while values below 0.2 suggest weak similarity [31]. For Jaccard similarity, values above 0.5 typically denote significant overlap between two texts, whereas values close to zero indicate minimal similarity [30]. Therefore, these results suggest that despite referring to the same vulnerability, the textual content of these reports from different databases is not similar. However, we also observed that while the overall similarity of the entire records is low, certain fields exhibit higher

[6] https://www.cve.org/CVERecord?id=CVE-2024-45039.
[7] https://osv.dev/vulnerability/GHSA-q3hw-3gm4-w5cr.



(a) Cosine Similarity (b) Jaccard Similarity

Fig. 2. The similarity distribution of duplicate vulnerability pairs.

similarity. For instance, as shown in Fig. 1, the "Title" field in the CVE record and the "Summary" field in the OSV record are more similar. In summary, the overall similarity of duplicate vulnerability pairs across different databases remains low.

Based on the results of our pilot study, it is safe to conclude that the sources of vulnerability data in different databases are diverse, as the records of duplicate vulnerabilities across these databases share limited common content. Despite this lack of shared information, database maintainers have identified and recognized these records as referring to the same vulnerabilities (e.g., by associating them with the same CVE ID). Currently, this identification process relies primarily on manual efforts by database maintainers. Identifying duplicate vulnerabilities across different databases, especially linking them to standard databases like CVE, is crucial. Users may receive different or conflicting details about the same vulnerability depending on which database they consult. Security teams may need to cross-reference multiple databases to obtain a complete picture of a vulnerability, leading to inefficiencies in the vulnerability management process. Detecting duplicate vulnerabilities can help create a more consistent and accurate representation of security issues across different databases, reducing the time and effort spent on analyzing what appear to be separate vulnerabilities but are actually the same issue.

However, identifying duplicate vulnerabilities across different databases is far from straightforward. Our pilot study indicates that the documentation of vulnerabilities varies substantially in both content and format, posing multiple challenges for accurate detection. Vulnerability records are often stored in JSON format and generally contain two main types of information: descriptive text and attribute fields. The descriptive text provides the principal semantic details of a vulnerability, whereas attribute fields (e.g., affected versions, product names, and package identifiers) typically consist of numerical data or brief phrases with limited semantic content. Due to these variations, some vulnerabilities feature comprehensive textual descriptions with minimal metadata, whereas others contain sparse textual details but extensive attribute fields. Selecting an appropriate detection strategy for such heterogeneous records is thus non-trivial. For instance, methods inspired by duplicate bug report detection—such as simple similarity-based approaches—rely on word frequency statistics while ignoring word order and context. Although this frequency-based approach is efficient and works effectively for concise descriptions with minimal variation, it may fail to capture essential similarities in longer, more diverse texts that differ in expression but describe the same vulnerability. Prior studies in duplicate bug report detection and issue tracking have similarly observed that lexical similarity techniques such as TF-IDF often struggle to identify semantically equivalent reports when there is substantial variation in wording or the presence of irrelevant content [32–34]. These limitations motivate the need for more robust similarity measures in our setting, where vulnerability descriptions across databases are often lengthy and inconsistently structured. On the other hand, deep learning-based methods can better capture the underlying semantics in lengthy or diverse descriptions, demonstrating clear advantages when handling complex texts. Nevertheless, these approaches face limitations when the descriptive text is brief, as insufficient context may hinder their ability to accurately grasp the complete semantics. Consequently, designing a detection method that is adaptable to different types of vulnerability records requires careful consideration. Furthermore, given the large volume of records within vulnerability databases, computational efficiency remains a crucial factor in ensuring the practicality of any proposed solution.

3. Dataset

The research on detecting duplicate vulnerabilities is still relatively limited, and as a result, there is currently no publicly available dataset specifically for this purpose. However, such a dataset is essential for advancing research in this area. A well-constructed duplicate vulnerability dataset would provide a reliable benchmark for evaluating and comparing different detection methods, helping to standardize approaches and measure progress in this field. It would also facilitate the development of more effective tools for detecting duplicates across various vulnerability databases, allowing researchers and practitioners to better understand the challenges involved and refine their techniques. In light of this need, we propose to construct such a dataset by analyzing and aggregating duplicate vulnerabilities from multiple well-known databases, making it publicly available to support further research and development.

To construct our dataset, we selected three authoritative vulnerability databases: OSV, CVE, and the GitHub Advisory Database. These databases are widely used, actively updated, and well-maintained, serving as benchmarks in the field of vulnerability infor-

mation. Since our objective is to identify duplicate vulnerabilities, determining which vulnerabilities are duplicates is essential—a task often facilitated by annotations from database maintainers. As discussed in Section 2, we observed that many other vulnerability databases reference CVE IDs. Maintainers often associate CVE IDs with vulnerabilities in their own databases to indicate duplicates of existing CVE entries. Beyond CVE IDs, in the open-source software domain, GHSA numbers from the GitHub Advisory Database serve as unique identifiers used to mark duplicate vulnerabilities across databases. For example, OSV vulnerabilities may reference several GHSA numbers to indicate which entries in the GitHub Advisory Database correspond to duplicates. This information is highly valuable and is manually verified and annotated by professional database maintainers, serving as a reliable gold standard for identifying duplicate vulnerabilities.

3.1. Cross-database duplicates

The first part of the dataset construction focuses on identifying duplicate vulnerabilities across three databases. These duplicates represent the same vulnerabilities reported in multiple databases, despite potential differences in their descriptions, formats, or metadata. Since duplicate vulnerabilities may exist between any two of the three databases, we systematically searched for marked duplicates in all possible pairwise combinations. Below, we describe the process for identifying duplicates between each pair of databases.

CVE and OSV: CVE, as a widely used standard vulnerability database, typically defines its own CVE IDs and does not reference IDs from other databases, such as the GHSA numbers from the GitHub Advisory Database. OSV, on the other hand, includes both CVE IDs and GHSA numbers. Therefore, to identify duplicates between these two databases, we relied solely on CVE IDs. As mentioned in Section 2, the "Aliases" attribute in OSV vulnerability records often contains references to corresponding CVE IDs, providing a direct and efficient method for identifying vulnerabilities shared between the two databases. To ensure accuracy, the extracted CVE IDs from OSV were cross-referenced with vulnerability entries in the CVE database. Due to the large volume of data, we restricted the analysis to vulnerabilities reported from 2021 to 2023. This approach resulted in 19,919 pairs of duplicate vulnerabilities between CVE and OSV, with 12,876 pairs from 2021, 3,505 pairs from 2022, and 3,538 pairs from 2023.

CVE and GitHub Advisory Database: Similar to the previous case, we relied on CVE IDs to identify duplicates between CVE and the GitHub Advisory Database. While many GitHub Advisory Database vulnerabilities include corresponding CVE IDs, not all do, as some vulnerabilities either lack a CVE entry or have not been annotated by the database maintainers. Again, we restricted our search to vulnerabilities reported from 2021 to 2023. This process identified 8,013 pairs of duplicate vulnerabilities between CVE and the GitHub Advisory Database.

OSV and GitHub Advisory Database: In this case, OSV vulnerabilities may reference both CVE IDs and GHSA numbers—sometimes multiple GHSA numbers—which allows us to use both identifiers to detect duplicates. We compared vulnerabilities from OSV and the GitHub Advisory Database reported from 2021 to 2023. If a vulnerability in OSV was associated with the same CVE ID or linked to the same GHSA number as a vulnerability in the GitHub Advisory Database, we considered them duplicates. This method yielded 5,042 pairs of duplicate vulnerabilities. Upon further investigation, we found that many of these duplicates were essentially identical records. For example, OSV entries often directly shared the same IDs as the corresponding GitHub Advisory Database entries, with nearly identical content. According to OSV's official documentation, the GitHub Advisory Database is a source for OSV, which explains the potential duplication between the two. As these duplicates represented identical records, we decided to remove them. After filtering, we retained 978 pairs of unique duplicate vulnerabilities between OSV and the GitHub Advisory Database.

It is important to note that removing duplicates between OSV and the GitHub Advisory Database may also impact previously identified duplicates between CVE and OSV, as well as between CVE and the GitHub Advisory Database. Since there may be copied vulnerabilities between OSV and the GitHub Advisory Database, the vulnerabilities corresponding to CVE in these two databases might also be copies of each other. To address this, we rechecked the duplicates found between CVE and the other two databases. When we detected overlapping records, we retained only one entry to avoid duplicate entries in our dataset. After applying the above deduplication and filtering process, we retained 22,145 unique duplicate vulnerability pairs across different databases. Fig. 3 shows a Venn diagram illustrating the distribution of these cross-database duplicate pairs from 2021 to 2023. The red areas indicate the pairs retained between each pair of databases. Among the 74,567 CVE entries published during this period, we retained 19,919 unique duplicate pairs with OSV and 1,248 with the GitHub Advisory Database. Additionally, from the 60,887 OSV entries and 20,223 GitHub Advisory Database entries, we retained 978 unique duplicate pairs. As our analysis focuses on pairwise duplication across databases, the figure reports the number of duplicate pairs retained between each pair of databases. These results indicate that a significant number of vulnerabilities are reported in multiple databases, with the highest overlap between CVE and OSV. This is likely because many OSV entries are directly associated with CVE records. The overlaps between OSV and the GitHub Advisory Database, as well as between CVE and the GitHub Advisory Database, appear smaller in our dataset. This is because we removed many identical records between OSV and the GitHub Advisory Database, as mentioned earlier.

3.2. GitHub advisory database

In the process of constructing the cross-database duplicate vulnerabilities dataset, we observed an interesting phenomenon: the GitHub Advisory Database contains some duplicate vulnerability records. The GitHub Advisory Database aggregates vulnerabilities primarily contributed by different developers. As a result, multiple developers may independently submit reports for the same vulnerability. However, the database is not always able to recognize these as duplicates, particularly when the reports differ significantly or are submitted under different GitHub projects. Consequently, these vulnerabilities are often assigned different GHSA numbers and

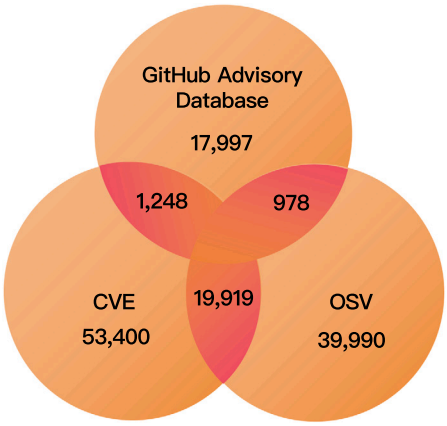


Fig. 3. Distribution of retained cross-database duplicate vulnerability pairs (2021–2023). (For interpretation of the colors in the figure(s), the reader is referred to the web version of this article.)

listed as separate entries in the database. Over time, vulnerability information in the GitHub Advisory Database is refined as maintainers cross-reference it with other databases, such as CVE. Many vulnerabilities in the GitHub Advisory Database are later associated with CVE IDs, and OSV entries may also link to vulnerabilities in the GitHub Advisory Database. We found that some vulnerabilities in the GitHub Advisory Database, each assigned different GHSA numbers, were subsequently linked to the same CVE ID. Similarly, vulnerabilities in OSV were often associated with multiple GitHub Advisory Database entries. These associations, manually verified by database maintainers, indicate that the linked vulnerabilities in the GitHub Advisory Database are actually duplicates.

To identify these duplicate vulnerabilities, we leveraged two sources of information: the "Aliases" attribute in the OSV database and the CVE IDs associated with vulnerabilities in the GitHub Advisory Database. Specifically, we analyzed each vulnerability report in the OSV database, inspecting the aliases attribute to check for references to GHSA numbers from the GitHub Advisory Database. If multiple GHSA numbers were listed under the aliases field, we flagged these as potential duplicates. Meanwhile, we examined the CVE IDs associated with vulnerabilities in the GitHub Advisory Database. If two distinct vulnerabilities in the GitHub Advisory Database shared the same CVE ID, we also flagged them as potential duplicates. For example, GHSA-7rjr-3q55-vv33[8] and GHSA-jfh8-c2jp-5v3q[9] are two separate advisories that both reference CVE-2021-44228, the well-known Log4Shell vulnerability. Despite the overlap in content, they are listed separately with different GHSA identifiers. Another example is GHSA-2m65-m22p-9wjw[10] and GHSA-vh55-786g-wjwj,[11] both of which correspond to CVE-2022-34716. Not only do these advisories reference the same CVE ID, but the GitHub Advisory Database also explicitly acknowledges in their descriptions that they refer to the same issue. These examples illustrate the patterns we used to identify duplicates. The complete set of such confirmed duplicates is included in our released dataset. This step focused on identifying duplicate vulnerabilities within the GitHub Advisory Database itself, complementing the cross-database dataset constructed earlier, which addresses duplicates across multiple databases.

It is worth noting that we did not find any inter-database duplicate vulnerabilities in CVE and OSV. Since CVE is a standardized and widely used database, it undergoes more rigorous verification, making duplicate entries between CVE and other databases rare. Also, we did not find inter-database duplicates using CVE IDs and GHSA numbers as identifiers. Therefore, this part of the dataset consists solely of duplicates within the GitHub Advisory Database. As a result, we successfully constructed 133 pairs of duplicate vulnerabilities within the GitHub Advisory Database. This subset of the dataset is crucial for evaluating methods that detect duplicates within a single data source, providing valuable insights into redundancy and consistency issues within a single database.

In conclusion, the final vulnerability duplicates dataset (22,278 pairs) is composed of two parts: 22,145 cross-database duplicates and 133 inter-database duplicates within the GitHub Advisory Database. Together, these two parts form the complete dataset used in this study. We made this dataset publicly available at https://zenodo.org/records/15234053.

4. Method

The identification of duplicate vulnerabilities primarily relies on the textual contents in vulnerability records within databases. Therefore, detecting duplicates can be framed as a text-matching problem, where the goal is to identify potential duplicates based on the similarities between vulnerability descriptions. While the problem of duplicate vulnerability detection has not been extensively studied, insights can be drawn from similar tasks, such as duplicate bug report detection. These related tasks employ a variety of techniques, which can be broadly categorized into simple text similarity measures, word embeddings, and more advanced methods based on deep learning or pre-trained language models. These approaches vary in complexity and computational cost. Simple methods

[8] https://github.com/advisories/GHSA-7rjr-3q55-vv33.
[9] https://github.com/advisories/GHSA-jfh8-c2jp-5v3q.
[10] https://github.com/advisories/GHSA-2m65-m22p-9wjw.
[11] https://github.com/advisories/GHSA-vh55-786g-wjwj.

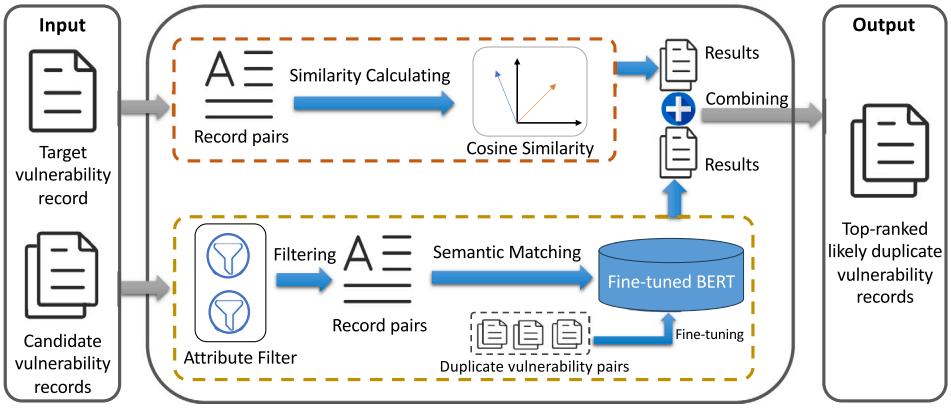


Fig. 4. The overview of our proposed method.

like cosine or Jaccard similarity are efficient but may miss nuanced semantic relationships. Word embeddings (e.g., Word2Vec) capture richer semantics but require pre-training. Advanced techniques, such as deep learning or BERT, offer contextual understanding but are computationally demanding and need large datasets. The challenge is selecting the most suitable approach for detecting duplicate vulnerabilities. Empirical studies [33,35] on duplicate bug report detection suggest that, in certain cases, simpler techniques (such as text similarity methods) can outperform more sophisticated approaches (e.g., deep learning-based methods), achieving comparable performance in real-world scenarios while being more efficient. Building on these insights, we propose a combined method for detecting duplicate vulnerabilities. This method aims to balance effectiveness and efficiency, leveraging the strengths of simpler techniques while maintaining computational feasibility for large-scale vulnerability databases.

4.1. Overview

Fig. 4 shows an overview of our proposed method. The input consists of a target vulnerability record and a set of candidate records, while the output is the top N most likely duplicate vulnerabilities, ranked by likelihood. Previous studies [33] on related tasks indicate that simpler techniques can achieve results comparable to those of more complex methods, yet with greater efficiency. Therefore, our method combines both simple techniques (e.g., cosine similarity) and more sophisticated methods (e.g., BERT-based approaches) to balance efficiency and effectiveness. The primary goal of our method is to handle the large-scale nature of vulnerability databases while maintaining identification accuracy. Cosine similarity offers computationally efficient screening when vulnerability records share similar structures and vocabulary. However, it can be less effective for more diverse texts that use different phrasings, prompting us to integrate cosine similarity with BERT-based methods. BERT excels at capturing contextual and semantic relationships between words, making it suitable for identifying duplicates not readily apparent through simple text matching. Nonetheless, applying a generic, unrefined BERT model to all candidate records can be inefficient and may lack domain knowledge. Consequently, we propose fine-tuning BERT on domain-specific data and filtering candidate vulnerabilities before passing them to the model, thereby improving efficiency. By combining these strategies, our method achieves both speed (via cosine similarity) and semantic accuracy (via BERT), offering a hybrid solution for duplicate vulnerability detection.

Hence, as illustrated in Fig. 4, our method is composed of two main components. The first component is a cosine similarity-based approach for duplicate vulnerability identification. In this step, we compare the target vulnerability record with each candidate record by considering not only the textual content in fields such as summary and description but also other relevant attributes (e.g., product and component). These attributes have been shown to be valuable for identifying duplicates [24], particularly when the textual description is limited and other attributes are available. From these comparisons, we obtain a similarity score between the target vulnerability record and each candidate. As noted earlier, cosine similarity can struggle when handling text with highly diverse expressions. Therefore, the second component is a BERT-based approach. Before comparing text, we apply an attribute filtering process (cf. Section 4.3), which reduces the candidate set by leveraging specific attribute values. We then fine-tune a BERT model on a large dataset of duplicate vulnerabilities. After fine-tuning, this model is used to match descriptive information from vulnerability records and produce its own similarity scores. Finally, we combine the results from both components. Since the cosine similarity-based approach and the fine-tuned BERT model both produce numeric scores, we add these scores together for each candidate and rank them accordingly. The top-ranked N vulnerabilities are deemed more likely to be duplicates by our method.

4.2. Cosine similarity-based matching

The first step of our method is to match the target vulnerability record with each candidate record to assess the likelihood of duplication. As discussed in Section 4.1, we begin with a cosine similarity-based approach due to its efficiency and effectiveness in processing large volumes of textual data. We selected cosine similarity for several reasons. First, it is computationally lightweight and scales well with large datasets, making it particularly suitable for screening large vulnerability databases. Second, many vulnerability records share a relatively structured format, where key information (e.g., affected product, version, and impact) often appears in

similar lexical patterns. In such cases, cosine similarity can effectively identify duplicates based on shared vocabulary and phrasing. Third, when applied to normalized and preprocessed text representations, cosine similarity captures the overall content similarity between records without requiring exact word matches, which is desirable in early-stage filtering.

To prepare for similarity calculation, we concatenate all available textual fields in a vulnerability report—excluding those that may directly reveal duplication relationships such as “Aliases”, “References”, or “CVSS” scores—into a single document. This design choice is motivated by the structural inconsistencies across different vulnerability databases: for example, the CVE database uses a “Product” field, while OSV may use a “Package” field to convey similar information. Rather than aligning such fields manually, we treat each report as a whole and unify its content into a single text. This allows us to make full use of the information present, especially when some fields are sparse or missing, and ensures that our approach is robust to schema variations. Prior work has also shown that attributes beyond just the description are valuable for identifying duplicates, particularly when textual descriptions are brief or ambiguous [24]. We then apply standard preprocessing steps, including lowercasing, stop word removal, and token normalization, and convert the resulting text into a term frequency-inverse document frequency (TF-IDF) weighted vector. We compute the cosine similarity between the target record and each candidate and store the resulting scores for further ranking. While cosine similarity works well for structurally similar records, it may underperform when texts use different phrasings to describe the same issue. To address this limitation, we incorporate a BERT-based matching method in the second component of our method. This hybrid strategy enables us to combine the scalability of cosine similarity with the semantic understanding provided by BERT, leading to more accurate duplicate detection overall.

Our method is particularly useful in scenarios where a new vulnerability has been submitted but has not yet been reviewed or linked by database maintainers. For example, before an OSV entry is associated with a CVE ID, our method can assist maintainers in quickly narrowing down potential duplicates. However, if a vulnerability has already been linked via fields such as “Aliases”, our method is not required. This targeted application ensures that our method supports efficient and timely vulnerability management.

4.3. Attribute filtering

Vulnerability records across different databases can vary considerably in both representation and attribute fields. For example, as shown in Fig. 1, CVE and OSV records differ substantially. In addition to descriptive text, each database may include attributes such as ecosystem, affected product/package names, versions, and references. These attributes often consist of numerical data or short phrases that may be incomplete, inconsistent, or non-standardized across databases. Moreover, when BERT-based approaches process these brief, fragmented attributes, they may introduce noise rather than contributing meaningful context. Consequently, we filter out these attributes when employing the BERT-based approach. By focusing on the shared descriptive information (e.g., vulnerability descriptions), the BERT component of our method reduces noise, handles missing data, and improves accuracy in detecting duplicates within heterogeneous datasets.

Furthermore, BERT-based matching is typically less efficient than cosine similarity for large-scale comparisons. Theoretically, every vulnerability record might be a duplicate candidate for a given target record, resulting in extensive and computationally expensive evaluations. While cosine similarity computations remain fast enough to apply to all possible candidates, relying on BERT in the same manner would significantly affect performance. To address this, we use a minimal set of attributes to balance narrowing the candidate set against the risk of excluding valid duplicates. This strategy not only helps BERT avoid potential distractions but also improves efficiency by reducing the volume of candidate records. In this initial screening, certain attributes—specifically Product/Vendor, Package, Version, and Reference—facilitate a first pass at selecting potentially related records by matching ecosystem or version details. We refine this subset using text-based fuzzy matching and similarity metrics, ensuring that minor naming inconsistencies do not conceal actual duplicates. To maintain flexibility, we apply “or” conditions across attributes, thus minimizing over-reliance on any single aspect. In addition, we address inconsistencies in Product/Vendor and Package attributes by applying lightweight normalization methods (e.g., standardized casing and removal of special characters), maintaining a minimal set of alias mappings, and using relaxed similarity thresholds. Through these steps, we reduce the candidate pool while minimizing the likelihood of discarding authentic duplicates.

These measures, primarily motivated by efficiency considerations, decrease the number of record pairs that must be evaluated using BERT. However, one potential drawback is that we may unintentionally remove attributes that could aid BERT in identifying duplicates or discard genuine candidates prematurely. Nevertheless, the cosine similarity component helps compensate for this limitation. Because cosine similarity evaluates all attributes and compares every candidate record to the target record, it can uncover relationships that may be overlooked by the BERT-based approach. This synergy between cosine similarity and BERT is a key advantage of our combined method. To further assess the impact of this filtering on the method, we conduct ablation experiments in Section 5.2.2, comparing the performance of the method with and without attribute filtering. These experiments provide a detailed discussion on how this strategy affects performance.

4.4. Fine-tuned BERT-based matching

Directly using BERT to identify duplicate vulnerability records may not yield optimal results, as BERT is pretrained on general text data and may not effectively capture the nuanced language specific to vulnerability descriptions. To address this limitation, we propose fine-tuning the BERT model using domain-specific vulnerability data. Fine-tuning enables the model to better understand and match the contextual and semantic nuances inherent in vulnerability records. To fine-tune BERT for more accurate duplicate vulnerability identification, we first require a substantial number of duplicate vulnerability records. In Section 3, we constructed a large-scale

dataset of duplicate vulnerabilities, which is well-suited for fine-tuning the BERT model. This dataset, encompassing duplicates identified over the past three years from three different databases, is both extensive and representative of current vulnerability data. Moreover, these vulnerabilities have been thoroughly documented, analyzed, and cross-referenced across multiple databases, ensuring a comprehensive and reliable resource for fine-tuning. In total, the dataset contains 22,278 pairs of duplicate vulnerabilities sourced from CVE, OSV, and the GitHub Advisory Database.

Note that, unlike the cosine similarity-based component, the input texts used for BERT fine-tuning are not preprocessed using traditional NLP steps such as lowercasing or stop word removal, in line with the original BERT design. To prevent the model from becoming biased toward predicting all pairs as duplicates, we also generated an equal number of negative examples through random matching. This approach ensures that the model learns to distinguish between matching and non-matching pairs, rather than merely memorizing specific patterns in positive examples. Consequently, our training dataset comprised 44,592 pairs of text data, which we split into 70% for training, 10% for validation, and 20% for testing. With the fine-tuned BERT model, we are now equipped to match target vulnerabilities with candidate vulnerabilities more accurately. This fine-tuned model leverages both the contextual understanding of BERT and the domain-specific knowledge acquired during fine-tuning.

Once the model is fine-tuned, it can be applied to match the target vulnerability record with the candidate vulnerability records. In our method, each of the two components produces a numeric score. To obtain a final ranking, we add these scores for each candidate and then select the top N results. The parameter N determines how many of the highest-ranked candidates are returned for human review. While N does not fundamentally alter the core detection process, it does affect the workload for vulnerability database maintainers. A smaller N reduces the number of records to be confirmed, thereby decreasing maintenance effort. However, setting N too low may mean that some valid duplicates are overlooked. In other words, N influences both the method's practicality and its overall effectiveness. In our experiments, we evaluated different values of N to identify an optimal balance between maintainer workload and the likelihood of capturing true duplicates.

5. Evaluation

In the study, we aim to address the following research questions:

- **RQ1 (Effectiveness):** How effectively does our proposed method identify duplicate vulnerabilities in the constructed dataset, and how does it compare to baseline methods?
- **RQ2 (Ablation Study):** How do the method's key components (i.e., attribute filtering and BERT-based matching) influence its effectiveness?
- **RQ3 (Practical Utility):** Can the proposed method assist maintainers in identifying duplicate vulnerabilities within the context of ongoing database maintenance?

These questions are designed to systematically evaluate both the technical performance and practical applicability of our method. RQ1 addresses the fundamental capability of the approach and benchmarks it against established techniques. RQ2 investigates how each component of the method contributes to its overall performance, guiding practitioners in refining the approach for different use cases. Finally, RQ3 considers the method's real-world utility, exploring whether it can enhance the workflow of database maintainers and contribute to more efficient vulnerability management.

5.1. Experimental setup

Next, we present the experimental setup, including the dataset, the baseline methods used for comparison, and evaluation metrics.

Dataset. We utilize the dataset introduced in Section 3 for evaluation. As described in Section 4.3, 70% and 10% of this dataset were allocated for BERT fine-tuning (training and validation, respectively). To avoid bias, we do not use these portions for the overall evaluation of our method. Instead, we rely on the remaining 20% of the dataset, which comprises 4,461 duplicate vulnerability pairs. This subset, though smaller, remains sufficiently large and representative for an effective evaluation, ensuring that our testing results are unbiased and robust.

Baseline methods. Since no dedicated methods currently exist for duplicate vulnerability detection, we establish baselines using approaches adapted from the closely related domain of duplicate bug report detection. This approach provides a fair foundation for comparison and allows us to understand how well these established techniques can be adapted to our setting, as well as how our method performs relative to them. We compare our method against three categories of baselines. The first is a text similarity approach (cosine similarity) that computes textual similarity scores between a target vulnerability record and all candidate records, ranking and returning the top matches based on these scores. The second and third categories both rely on BERT. The second applies BERT directly without any domain-specific fine-tuning, whereas the third employs a BERT model fine-tuned on vulnerability-related data (as described in Section 4.4), enabling it to capture domain-specific nuances. However, these two approaches only utilize BERT. In RQ2, we further explore and compare methods based on other models. By comparing our proposed method to these baseline methods, we can thoroughly evaluate its effectiveness in detecting duplicate vulnerabilities. This comparison also reveals the strengths and limitations of existing techniques when applied to our task, providing insights into how they may be refined or extended in future work.

Metrics. To evaluate the effectiveness of different methods in identifying duplicate vulnerability records, we employ evaluation metrics commonly used in ranking-based tasks such as duplicate bug report detection and information retrieval. Specifically, we use

Table 1
Results of our method and the compared baseline methods.

Method	Hit Rate@N				MRR	Mean	Median
	1	3	5	10			
Ours	0.85	0.92	0.93	0.95	0.97	1.11	1
Cosine	0.74	0.83	0.85	0.87	0.90	1.38	1
FT-BERT	0.42	0.50	0.53	0.58	0.80	1.9	1
BERT	0.19	0.33	0.37	0.42	0.64	2.6	2

Note: "Ours" denotes our proposed method, while "Cosine", "BERT", and "FT-BERT" represent baseline methods using cosine similarity, pre-trained BERT, and fine-tuned BERT, respectively. The table shows Hit Rate@N ($N = 1$, 3, 5, and 10), MRR, Mean Rank, and Median Rank.

Hit Rate@N, *Mean Reciprocal Rank* (MRR), *Mean Rank*, and *Median Rank*. These metrics are well aligned with our problem setting, where the goal is to rank the true duplicate as highly as possible among a list of candidates. *Hit Rate@N* assesses whether at least one correct duplicate appears among the top *N* results, reflecting the method's ability to surface relevant items in prominent positions. *MRR* captures how early the first relevant result is retrieved across queries, providing an intuitive measure of ranking quality when only one correct answer exists. Since our method returns the top *N* results for each target, MRR is particularly suitable for assessing its performance. *Mean Rank* and *Median Rank* further evaluate the average and median positions of the correctly identified duplicates in the returned lists, offering additional insights into the overall ranking behavior. We also considered metrics such as Precision (or Precision@N), Recall (or Recall@N), F1 score, and ROC-AUC. However, these metrics are generally more suitable for binary classification tasks, where each candidate is independently classified as positive or negative. In our setting, each target vulnerability is typically associated with only one known duplicate in the ground truth, as we focus on duplicate pairs. Consequently, Recall@N becomes effectively equivalent to Hit Rate@N (i.e., whether the known duplicate is ranked within the top *N*), while Precision@N becomes less informative, especially when multiple candidates are returned but only one is labeled as relevant. Metrics such as F1 score and ROC-AUC, which aggregate performance across binary decisions, are not directly applicable to a ranked retrieval task like ours. Therefore, we adopt ranking-based metrics that more accurately reflect the goal and nature of our method.

5.2. Experimental design and results

In the following, we present a detailed description of the experimental designs and their corresponding results for each research question.

5.2.1. RQ1: effectiveness

To address RQ1, we compared our method with the baseline methods introduced in the previous subsection using the test dataset. For each duplicate vulnerability pair in the test set, we randomly selected one record as the target and treated all remaining records in the dataset as candidates. The goal was to determine whether each method could accurately identify the corresponding duplicate vulnerability among these candidate records. Since the ground truth duplicate relationships in our dataset were derived from CVE IDs or GHSA numbers—often manually associated by database maintainers after the initial publication—we designed the experiment to simulate an earlier point in time, before such associations were established. To do so, we carefully reconstructed the initial state of each record as it would have appeared when first reported, which aligns with the practical usage scenario of our method. The records in our test set come from CVE, OSV, and the GitHub Advisory database. For GitHub Advisory records, we recovered the initial versions by analyzing the commit history of the advisory repository. In cases where records were imported in bulk without detailed history, we manually traced back to the relevant commits in the original GitHub repositories. For CVE entries, if a record had not been updated since its publication, we treated its current version as the initial one. For updated records, we used the CVEProject's cvelist repository,[12] which logs each CVE's creation and modification history as Git commits, allowing us to retrieve the earliest version of the record. As for OSV entries, although only a single "modified" timestamp is available on the official website, we were able to obtain the original versions for some records by inspecting associated GitHub projects. In all cases, we removed potentially revealing fields (e.g., "Aliases" or "CVSS" scores) that could explicitly indicate duplication. All methods, including baselines, were evaluated under the same controlled conditions to ensure fairness.

In addition to the overall evaluation, we further partitioned the test dataset based on the reported dates of the vulnerabilities, in response to the reviewer's concern. Since our dataset spans from 2021 to 2023, we divided it into three subsets by year and conducted experiments separately on each. Specifically, the test set contains 8,918 duplicate pairs in total, with 5,350 pairs from 2021, 1,783 from 2022, and 1,785 from 2023. This temporal division enables us to examine whether the performance of our method and the baselines changes over time, providing a more comprehensive evaluation of their robustness in different reporting periods.

Results. Table 1 presents the results of our proposed method compared to the baseline methods on the test set that combines data from 2021 to 2023. As shown, our method consistently outperforms all baselines across all metrics, including Hit Rate@N, MRR, Mean Rank, and Median Rank. Columns two through five display the Hit Rate@N for $N = 1$, 3, 5, and 10. Hit Rate@N represents

[12] https://github.com/CVEProject/cvelist.

Table 2
Results of our method and baseline methods for the 2021 duplicate vulnerability.

Method	Hit Rate@N				MRR	Mean	Median
	1	3	5	10			
Ours	0.84	0.91	0.92	0.94	0.96	1.13	1
Cosine	0.73	0.82	0.84	0.86	0.89	1.39	1
FT-BERT	0.41	0.49	0.52	0.57	0.79	1.92	1
BERT	0.18	0.32	0.36	0.41	0.63	2.75	2

Table 3
Results of our method and baseline methods for the 2022 duplicate vulnerability.

Method	Hit Rate@N				MRR	Mean	Median
	1	3	5	10			
Ours	0.86	0.93	0.94	0.96	0.97	1.1	1
Cosine	0.75	0.84	0.85	0.88	0.91	1.36	1
FT-BERT	0.43	0.50	0.54	0.59	0.82	1.85	1
BERT	0.20	0.34	0.38	0.44	0.65	2.55	2

Table 4
Results of our method and baseline methods for the 2023 duplicate vulnerability.

Method	Hit Rate@N				MRR	Mean	Median
	1	3	5	10			
Ours	0.85	0.92	0.93	0.95	0.97	1.11	1
Cosine	0.74	0.83	0.85	0.87	0.90	1.38	1
FT-BERT	0.42	0.50	0.53	0.58	0.80	1.9	1
BERT	0.19	0.33	0.37	0.42	0.64	2.62	2

the proportion of target vulnerabilities for which the correct duplicate appears among the top N results. In this task, each target vulnerability has only one correct duplicate record. Notably, our method achieves an 85% Hit Rate@1, indicating that 85% of correct duplicates are ranked first. When $N = 3$, the Hit Rate@3 increases to 92%. These results are further supported by the high MRR (0.97), as well as the favorable Mean Rank (1.11) and Median Rank (1). In general, a high MRR indicates that correct results are often found at top positions, and lower Mean/Median Ranks imply that the correct records rank closer to the first position. Among the baselines, the cosine similarity-based method achieves reasonably good performance, particularly in Hit Rate@N and MRR. This may limit the room for further absolute improvement. However, our method still improves upon it across all metrics. Although some improvements may appear moderate in magnitude (e.g., 0.93 vs. 0.85 in Hit Rate@5), they can lead to meaningful practical gains. For example, increasing Hit Rate@5 from 0.85 to 0.93 reduces the expected number of missed duplicates in every 100 cases from approximately 14 to 7, thereby reducing the effort required for manual verification. Compared to the other baselines, our method shows larger improvements over both the pre-trained and fine-tuned BERT models. This is consistent with findings in prior research [33,35], which note that simpler textual similarity methods can sometimes outperform more complex deep learning models when not properly adapted. We also observe a substantial performance gain when using fine-tuned BERT compared to its non-fine-tuned counterpart, highlighting the importance of fine-tuning in BERT-based approaches.

In terms of computational cost, we note that the fine-tuning of the BERT model required approximately 20 hours in our experiments. However, this is a one-time offline process. For inference, the average prediction time per query (i.e., for identifying the top candidates for a given vulnerability) was approximately 10 seconds for cosine similarity, 12 seconds for the BERT model, 12 seconds for the fine-tuned BERT model, and 22 seconds for our method. While BERT-based methods are slower than simple similarity measures, the prediction times remain within a practical range for real-world use. In particular, the end-to-end inference time is sufficiently fast for most offline analysis scenarios or batch processing pipelines. Therefore, the performance gains offered by incorporating BERT justify the additional computational cost in our target use cases.

We further evaluated the performance of our method and the baselines on temporally partitioned subsets of the test set, divided by vulnerability reporting year as previously described. This analysis assesses robustness over time. As shown in Tables 2 to 4, the results across yearly subsets are consistent with the overall trends, demonstrating the stable performance of our method. Our method consistently achieves high Hit Rate@N and MRR, along with favorable Mean and Median Ranks, outperforming all baselines each year. For instance, on the 2021 subset (Table 2), it achieves a Hit Rate@1 of 0.84, MRR of 0.96, Mean Rank of 1.13, and Median Rank of 1. The cosine-based method performs reasonably well (Hit Rate@1: 0.73, MRR: 0.89), while BERT-based baselines, especially without fine-tuning, perform worse. The 2022 and 2023 results (Tables 3 and 4) show similar patterns. Our method maintains a clear lead across all metrics (e.g., Hit Rate@1 of 0.86 and 0.85; MRR of 0.97 in both years). Although the cosine-based baseline

remains competitive, our method consistently delivers better results. Fine-tuned BERT variants improve slightly but still lag behind. These results highlight the robustness and generalizability of our method across different reporting years, with minimal impact from temporal variation.

Additionally, we conducted an in-depth analysis of the cases where either the cosine similarity-based method or the fine-tuned BERT-based method successfully identified duplicate vulnerabilities. This case-level examination offers insights into the complementary strengths of the two approaches and provides further evidence for the effectiveness of combining them. Although cosine similarity consistently outperforms other baseline methods on average, there are instances in which the fine-tuned BERT model correctly identifies duplicates that cosine similarity fails to detect. In such cases, the average cosine similarity scores are relatively low: 0.1549 for the descriptive text and 0.0783 for the full text. These records describe the same vulnerability but exhibit minimal lexical overlap, making them challenging for term-frequency-based methods to associate. For example, the CVE record CVE-2022-20142[13] and its corresponding OSV entry ASB-A-216631962[14] refer to the same Android security vulnerability involving improper access control in the telecom component. However, the textual descriptions differ significantly in surface form. While the CVE record uses relatively concise language focused on the impact, the OSV entry provides broader contextual details, including references to patch levels and internal tracking identifiers. Despite these lexical differences, BERT is able to associate the two based on their underlying semantic similarity.

Our findings further reveal that cosine similarity, which relies on bag-of-words representations and term frequency statistics, performs well when vulnerability descriptions are concise and exhibit a clear overlap in terminology. However, its effectiveness diminishes when applied to more verbose or structurally varied texts. In such situations, BERT's ability to capture context-dependent semantics enables it to detect latent relationships that are not easily identifiable through surface-level analysis. Nonetheless, BERT is not without limitations. When the available descriptive text is minimal or highly domain-specific (e.g., containing code snippets or references without sufficient context), its performance may suffer due to the lack of semantic cues. In contrast, cosine similarity can sometimes compensate for this by incorporating auxiliary fields, such as product names or affected versions, into the similarity computation. For instance, in the duplicate pair CVE-2022-35964[15] and GHSA-f7r5-q7cx-h668,[16] the OSV record includes partial code fragments and additional technical context that diverge considerably from the more abstract CVE description. These differences make it difficult for BERT to recognize the semantic equivalence between the two. However, cosine similarity, aided by structured metadata, is able to surface the correct match from a large candidate pool.

These observations underscore the advantages of a hybrid approach that leverages both cosine similarity and BERT. Given the heterogeneity of vulnerability records—ranging from sparse metadata to richly detailed narratives—no single method is universally effective. Cosine similarity provides an efficient baseline for shorter, well-aligned texts, while BERT offers robustness in capturing deeper semantic relationships. By integrating the two, our method benefits from their respective strengths, resulting in a more comprehensive and reliable solution for duplicate vulnerability detection. This combined approach also helps mitigate false negatives, as each method compensates for the other's weaknesses. Previous studies on duplicate bug report detection have compared similar methods but typically only reported outcomes that varied depending on the datasets. Our findings indicate that these discrepancies largely arise from the unique characteristics of the problems at hand, emphasizing the importance of adapting methods to different types of data and contexts.

5.2.2. RQ2: ablation study

To address RQ2, we conducted a series of studies to examine how key components affect our method's effectiveness. First, we performed ablation studies to evaluate the roles of attribute filtering and BERT-based matching. Removing the attribute filtering step allowed us to assess whether this noise-reduction and efficiency-enhancing process influences performance. Notably, without attribute filtering, all candidate vulnerability records are compared to the target record using the fine-tuned BERT model—a process that, as previously mentioned, can be inefficient. Therefore, we carried out the experiment on 50% of the test set for practical considerations, which still provided a representative sample. Moreover, the results with attribute filtering on this portion of the test set were consistent with those in RQ1, demonstrating the viability of using a reduced test set for this experiment. Additionally, we replaced the BERT-based matching component with alternative language models (e.g., T5 [36], RoBERTa [37], DeBerta [38], and ELECTRA [39]) to ascertain whether the observed improvements are unique to BERT or extend to other advanced models. Together, these experiments provide insights into the method's adaptability and robustness, assisting practitioners in choosing parameter settings and model configurations that best align with their operational needs.

Results. The attribute filter is specifically applied to the fine-tuned BERT-based matching component of our method, with two main objectives: to remove potentially misleading attributes that could negatively affect BERT's semantic understanding, and to reduce the number of candidate vulnerability records for improved efficiency. This filtering step directly impacts both the accuracy and speed of the BERT-based matching process. To assess its effect, we compare the performance of the matching component with and without the attribute filter, as presented in Table 5. The results show that while Hit Rate@N is marginally higher without the filter—likely due to the increased number of candidates improving the chance of retaining correct duplicates—other metrics, including MRR, Mean Rank, and Median Rank, remain largely unaffected. These findings suggest that the filter has a limited impact on overall effectiveness.

[13] https://www.cve.org/CVERecord?id=CVE-2022-20142.
[14] https://osv.dev/vulnerability/ASB-A-216631962.
[15] https://www.cve.org/CVERecord?id=CVE-2022-35964.
[16] https://osv.dev/vulnerability/GHSA-f7r5-q7cx-h668.

Table 5
Results of the BERT-based matching component with and without the attribute filter.

Method	Hit Rate@N				MRR	Mean	Median	Time
	1	3	5	10				
With Filter	0.42	0.51	0.54	0.58	0.82	1.78	1	0.5 h
Without Filter	0.47	0.54	0.56	0.63	0.82	1.94	1	5 h

Table 6
The overall method's results with and without the attribute filter.

Method	Hit Rate@N				MRR	Mean	Median	Time
	1	3	5	10				
With Filter	0.89	0.96	0.96	0.97	0.96	1.12	1	1 h
Without Filter	0.89	0.95	0.96	0.96	0.96	1.12	1	8 h

Nevertheless, we observed that the filter can lead to occasional misses of valid duplicates. In our evaluation, the proportion of missed cases due to filtering is approximately 5%. This typically occurs when the attribute values used for filtering are inconsistent between duplicate records. For example, in the duplicate pair CVE-2023-2136[17] and ASB-A-278113033,[18] the candidate was excluded because of mismatched affected product and version information. Additionally, when certain reports lack key fields (e.g., affected product), they may be unintentionally filtered out, reducing the chance of successful matching. Despite this, the performance trade-off appears to be justified. The attribute filter substantially improves processing efficiency: the total processing time is reduced from roughly 5 hours to 0.5 hours. Given that the accuracy degradation is relatively small and the speedup is significant, we believe this strategy provides a reasonable balance between effectiveness and efficiency, particularly in practical large-scale settings.

Furthermore, we evaluated the overall performance of our entire method with and without the attribute filter, as shown in Table 6. The results indicate that the overall ability to identify duplicate vulnerability records remains virtually unchanged whether the attribute filter is used or not. This consistency is attributed to the complementary nature of the cosine similarity and BERT-based matching components. While removing the attribute filter slightly diminishes the performance of the BERT-based matching, the cosine similarity component compensates by evaluating all attributes and identifying duplicates that BERT might miss. Consequently, the combined method maintains robust performance even with the attribute filter applied. Additionally, the overall efficiency of the method is significantly improved when using the attribute filter, making it a necessary component for practical deployment. In summary, the attribute filter optimizes our method by reducing computational overhead and minimizing noise without adversely affecting the accuracy of duplicate vulnerability detection. The complementary strengths of the cosine similarity and BERT-based matching components ensure that the overall method remains effective and efficient, making the attribute filter an essential element of our method.

Additionally, we conducted an experimental evaluation of the BERT model used in our method to assess the impact of alternative language models. Specifically, during the fine-tuning process, we replaced BERT with other advanced models such as RoBERTa, T5, DeBERTa, and ELECTRA. Fig. 5 illustrates the performance of these models on the test dataset, evaluating metrics including Accuracy, Recall, Precision, and F1-score. Overall, BERT achieved the highest performance among the five models, securing the best scores in three out of four metrics. RoBERTa was the second most effective, performing well across multiple metrics. The remaining models exhibited varying strengths and weaknesses. For example, T5 excelled in Precision but lagged in Recall, while DeBERTa and ELECTRA showed mixed results depending on the specific metric evaluated. These differences can be attributed to the distinct architectural features and pre-training objectives of each model, which influence their ability to capture and interpret semantic nuances in vulnerability descriptions. The superior performance of BERT underscores its effectiveness in our duplicate vulnerability record detection task, thereby validating our choice of this model.

5.2.3. RQ3: practical utility

To evaluate RQ3, we designed our experiments to reflect real-world scenarios where maintainers continuously update and expand their vulnerability databases. Our primary dataset, constructed from records spanning 2021 to 2023, was instrumental for training and validating the method. However, to assess the method's applicability in ongoing database maintenance, we needed to test it on more recent vulnerabilities that maintainers are currently processing. To this end, we collected new vulnerability records from CVE, OSV, and the GitHub Advisory Database, covering the period from January 2024 up to the time of writing. Among these recent vulnerabilities, we identified those that were not initially associated with any known duplicates (i.e., they did not have a linked CVE ID or GHSA number at the time of their initial submission) but have since been retrospectively confirmed as duplicates by database maintainers. By using these vulnerabilities as targets, we simulate a scenario where, during the ongoing maintenance phase, the method must identify duplicates before such associations are established. This design choice allows us to evaluate whether the method could have provided timely assistance to maintainers, helping them detect duplicate vulnerabilities more efficiently and reducing the

[17] https://www.cve.org/CVERecord?id=CVE-2023-2136.
[18] https://osv.dev/vulnerability/ASB-A-278113033.



(a) Accuracy (b) Recall

(c) Precision (d) F1-score

Fig. 5. Experimental performance results of different models on the test set.

Table 7
Results of our method and the compared baseline methods.

Method	Hit Rate@N				MRR	Mean	Median
	1	3	5	10			
Ours	0.74	0.91	0.93	0.94	0.88	1.32	1
Cosine	0.55	0.81	0.83	0.84	0.81	1.5	1
FT-BERT	0.54	0.59	0.60	0.62	0.91	1.38	1
BERT	0.11	0.14	0.15	0.17	0.73	2.3	1

reliance on manual verification processes that often occur later. In other words, we are testing the method's ability to function as an early-stage aid in the vulnerability management workflow, ensuring that it aligns well with the dynamic and continuously evolving nature of real-world vulnerability databases.

Results. From early 2024 to the time of writing, we identified 2,126 pairs of duplicate vulnerability records that meet the criteria outlined in our experimental design. These records were not included in the training phase of our method to ensure an unbiased evaluation. Table 7 presents the experimental results of our method compared to several baseline methods on this new 2024 dataset. The objective of this evaluation is to assess our method's capability to identify duplicate vulnerabilities within the context of ongoing database maintenance. The results indicate that our method's performance on the new dataset is consistent with its performance on the original test dataset. The primary difference is observed in the Hit Rate@1 metric, where our method demonstrates slightly lower performance on the 2024 data. However, as the value of N increases (e.g., N = 3,5,10), the differences in performance become negligible. Other metrics, such as MRR, Mean Rank, and Median Rank, also exhibit minimal variations, further demonstrating the robustness of our method. These findings highlight our method's ability to generalize effectively, confirming its capacity to handle unseen data and adapt to real-world database maintenance scenarios.

In contrast, the baseline methods exhibited varying performance on the new dataset. Notably, the fine-tuned BERT-based method maintained relatively stable performance, whereas the non-fine-tuned BERT method and the cosine similarity-based method showed reduced effectiveness. This discrepancy can be attributed to the nature of the new dataset, which likely includes more diverse textual descriptions and metadata, reflecting the dynamic and evolving nature of vulnerability reporting. The fine-tuned BERT model, trained on domain-specific data, is better equipped to capture the semantic nuances of these vulnerabilities. In contrast, methods relying solely on generic pre-trained models or simple textual similarity measures may struggle to accommodate this increased variability, resulting in decreased performance. The superior performance of our method is attributed to its hybrid design, which integrates the

efficiency of cosine similarity with the semantic depth of a fine-tuned BERT model. This combination allows our method to leverage the strengths of both approaches, ensuring high accuracy and computational efficiency. In summary, the results demonstrate that our method provides a practical and reliable solution for duplicate vulnerability detection in ongoing database maintenance. Its ability to generalize to unseen data, combined with the balanced integration of efficient and advanced techniques, underscores the effectiveness and adaptability of our method.

6. Threats to validity

In this study, we identify four types of threats to the validity of our findings: construct validity, internal validity, external validity, and conclusion validity. To ensure the reliability and generalizability of our results, we address these threats systematically across each dimension.

Construct Validity Threats: Construct validity concerns whether our study accurately measures the concept of duplicate vulnerability detection as intended. One potential threat is the assumption that textual similarity and shared attributes sufficiently capture all forms of duplicate vulnerabilities. There may be nuanced duplicates that differ significantly in their descriptions or attribute formats but still refer to the same underlying vulnerability. To mitigate this threat, we carefully defined the criteria for duplicate vulnerabilities based on comprehensive domain knowledge and ensured that our similarity measures encompass both lexical and semantic aspects. Additionally, by incorporating both cosine similarity and a fine-tuned BERT model, we aimed to capture a broader range of duplicate instances, addressing potential gaps in purely text-based or attribute-based approaches.

Internal Validity Threats: Internal validity pertains to whether the observed effects can be attributed to our proposed method rather than other confounding factors. One threat is the potential bias in dataset construction, as the identification of duplicate vulnerability records relied on manual annotation and cross-referencing across databases, which may introduce subjective judgments. Another threat is the possibility of overfitting the BERT model to our specific dataset during the fine-tuning process. To reduce bias in dataset construction, we employed objective criteria based on CVE IDs and GHSA numbers to identify duplicate records, ensuring consistency and reproducibility. Furthermore, we excluded the identified duplicate pairs from the training phase to prevent overfitting and to maintain the integrity of our evaluation. Cross-validation techniques and the use of separate training and testing datasets also helped ensure that our findings were not unduly influenced by specific data characteristics.

External Validity Threats: External validity concerns the extent to which our findings can be generalized to other settings, populations, or vulnerability databases. A potential threat is that our method was primarily evaluated on the CVE, OSV, and GitHub Advisory Database, which may not represent the full diversity of vulnerability databases in terms of structure, content, and reporting practices. To enhance external validity, we constructed and publicly released a comprehensive dataset encompassing multiple years and sources, thereby increasing the diversity and representativeness of our evaluation. Additionally, by testing our method on a newly identified set of duplicate records from early 2024, we demonstrated its applicability to ongoing database maintenance scenarios, suggesting that our method can generalize beyond the initial datasets used in training and testing.

Conclusion Validity Threats: Conclusion validity involves the appropriateness of the inferences drawn from our data analysis. A threat here is the reliance on specific performance metrics (e.g., Hit Rate@N, MRR, Mean Rank, Median Rank) that may not capture all dimensions of duplicate detection effectiveness. Additionally, the statistical significance of our results could be influenced by the size and distribution of our test datasets. To address these concerns, we employed a comprehensive set of evaluation metrics that collectively provide a holistic view of our method's performance. Hit Rate@N measures the ability to retrieve correct duplicates within the top N results, while MRR, Mean Rank, and Median Rank offer insights into the ranking quality of the detected duplicates. By analyzing multiple metrics, we ensured that our conclusions were robust and not dependent on a single aspect of performance. Moreover, conducting experiments on both the original and new datasets allowed us to verify the consistency and reliability of our findings across different data distributions.

By systematically identifying and addressing potential threats to validity across construct, internal, external, and conclusion dimensions, we have strengthened the reliability and applicability of our study. These measures ensure that our method for detecting duplicate vulnerabilities is both accurate and generalizable, providing a solid foundation for future research and practical implementation in vulnerability database maintenance.

7. Discussion

Our work is motivated by the observation that duplicate vulnerability records frequently appear across multiple security databases, yet identifying and linking these records remains a largely manual task. Developers consulting different sources—such as CVE and OSV—may encounter the same underlying issue under different identifiers and textual descriptions, resulting in confusion and duplicated effort. While the CVE system serves as a central reference point that other databases (e.g., OSV) may cite through fields like "Aliases", such links are often established only after manual verification and may be delayed well beyond the initial publication. Automating the detection and alignment of these duplicates has the potential to streamline vulnerability management workflows, reduce human effort, and improve consistency across security tools. However, our method currently relies on existing CVE IDs and GHSA numbers as ground truth, and does not address vulnerabilities that have not been assigned a canonical reference.

Large language model (LLM)–based embeddings are a promising alternative for this task. Models such as GPT-4.5 are capable of capturing rich semantic relationships across vulnerability descriptions—as illustrated in our qualitative example, where the model successfully linked several CVE IDs (e.g., CVE-2024-50354, CVE-2024-45039) to an OSV record shown in Fig. 1. However, in practice, several factors led us to adopt a fine-tuned BERT-based model instead. First, performing exhaustive pairwise comparisons among tens

of thousands of vulnerability records incurs substantial computational costs when using large LLMs. Second, retrospective evaluations using general-purpose LLMs risk leakage, as the model may have been trained on data that includes historical vulnerability entries. Nevertheless, in prospective settings—where only newly published vulnerabilities are matched—LLM embeddings could offer improved recall, particularly for subtle or obfuscated duplicates. Future work may explore hybrid methods that combine efficient filtering with LLM-based semantic matching for higher precision and scalability.

In our evaluation against multiple baselines, while the absolute performance gains over some methods, such as a cosine similarity baseline, may appear modest, this is partly because certain baselines already achieve reasonably strong results. Nonetheless, even incremental improvements can lead to meaningful benefits in practice. For instance, increasing Hit Rate@5 from 0.85 to 0.93 reduces the chance of missing a correct duplicate in the top five suggestions from one in seven cases to one in fourteen—potentially easing the burden of manual verification. In addition, our method improves Hit Rate@1 from 0.19 and 0.42 (achieved by BERT and our fine-tuned BERT model, respectively) to 0.85. These consistent improvements across all metrics underscore the practical value of our method in real-world scenarios involving vulnerability triage and database deduplication.

Finally, we acknowledge potential biases in our dataset of duplicate pairs, which were extracted from CVE, OSV, and GitHub Advisory Database records between 2021 and 2023. The dataset reflects the reporting conventions and linking practices of these databases, which may result in overrepresentation of certain vulnerability types and underrepresentation of others. Such imbalances may cause the model to overfit common patterns while underperforming on rare cases. In addition, our reliance on CVE and GHSA identifiers as ground truth assumes timely and consistent linkage across databases, yet our pilot study revealed that over half of OSV entries were associated with a CVE ID only after initial publication. This introduces label noise and temporal inconsistency. Future work could mitigate these issues by incorporating richer metadata, applying stratified sampling or data augmentation, or collaborating directly with database maintainers to curate more balanced and reliable duplication labels.

8. Related work

In recent years, a substantial body of research related to software vulnerabilities has emerged, covering topics ranging from vulnerability detection to in-depth vulnerability analysis. This section focuses on reviewing studies more closely aligned with our research objectives.

8.1. Recurring vulnerability detection

Our goal is to identify duplicate vulnerabilities in vulnerability databases, reducing maintainers' manual workload and providing earlier insights for database users. The closest research area to our objective is recurring vulnerability detection, which focuses on vulnerabilities that resurface across multiple software systems or versions. These issues often come from code reuse, cloning, branching, or reliance on common APIs and share patterns in code fragments, function calls, variable usage, or API interactions. They persist because underlying vulnerabilities remain unpatched or only partially addressed, allowing them to appear repeatedly in related codebases and diverse contexts. Over time, researchers have developed a range of techniques to detect recurring vulnerabilities at the code level. Early studies relied on abstract syntax trees and graph-based similarity measures [12] to identify common patterns in source code. Later approaches introduced code similarity techniques leveraging features derived from known patches and vulnerable segments [13], evolving to handle complex mutation patterns and distinguish vulnerabilities from their patched variants. Techniques such as lightweight function fingerprinting capture essential syntactic and semantic features, as well as patch characteristics, enabling efficient searches for previously known vulnerable functions in new codebases [14]. Program slicing methods refine these capabilities by extracting both vulnerability and patch signatures at syntactic and semantic levels [40], while signature-based static analysis and taint analysis offer systematic strategies for detecting patterns in large code repositories [41]. More recently, scalable methods employing multi-stage filtering and differential taint paths have effectively handled larger codebases [42]. Beyond code-level approaches, other studies detect recurring vulnerabilities at the binary level [43], or target specific domains, such as IoT firmware [44] and PHP web applications [45].

While these studies focus on detecting recurring vulnerabilities through structured code representations, patches, and semantic code features, our research addresses a different facet of the problem. Rather than identifying recurring vulnerabilities at the source or binary code level, we aim to detect duplicate vulnerabilities in vulnerability databases. Here, the emphasis is not on recognizing code-level patterns but on determining whether different databases describe the same underlying vulnerability. This shift introduces unique challenges, such as handling unstructured textual descriptions, varying attribute formats, and the absence of direct code-level indicators. In this manner, our approach complements existing code-centric recurring vulnerability detection methods by improving data consistency and reducing manual effort at the database level, ultimately contributing to more efficient and reliable vulnerability management.

8.2. Vulnerability detection and analysis

Beyond the detection of recurring vulnerabilities, a substantial body of research has addressed vulnerability detection in a more general context. Early efforts often employed program analysis techniques. For instance, Wang et al. [46] integrated constraint-based analysis and model checking to verify whether the memory constraints at certain program points could be violated, effectively converting the detection task into a reachability verification problem for specific assertions. Similarly, a variety of static analysis frameworks (e.g., Infer, Clang Static Analyzer, SVF, Fortify) have been developed to detect vulnerabilities such as buffer overflows,

memory errors, and information leaks. These tools rely on predefined rules or patterns, and their effectiveness depends on the comprehensiveness and accuracy of those rules. Empirical studies, such as [6], have examined the role and impact of these program analysis tools in industrial contexts. Similarity-based methods, which identify vulnerabilities by comparing new code fragments against known vulnerable patterns, have also been applied to general vulnerability detection scenarios [47,48].

In recent years, machine learning-based approaches have gained prominence in vulnerability detection, as evidenced by numerous studies, surveys [49,50], and empirical evaluations [51,52]. Although we do not aim to provide a comprehensive overview of these works here, readers are encouraged to consult the cited surveys for further details. While certain methods rely on traditional machine learning models (e.g., Random Forest, Naive Bayes) [7], the majority employ deep learning architectures, including recurrent neural networks [8], graph neural networks [9,10], and Transformer-based models [11]. These techniques typically employ source code representations or code-derived graphs/tokens, necessitating large, high-quality annotated datasets. To address limited labeled data, researchers have explored semi-supervised learning approaches, including pseudo-labeling [53]. Meanwhile, the integration of large language models and code intelligence techniques has further advanced the state of the art in vulnerability detection [54]. Beyond detection, other related works focus on vulnerability repair [15,16], impact analysis [17–19], and evolution analysis [20], examining how vulnerabilities propagate, change, and can be mitigated over time. While these works enhance our understanding of vulnerabilities from a code-centric or machine learning-driven detection perspective, our study differs in both objectives and methodology. Rather than focusing on code-level analysis or model-driven detection of new vulnerabilities, we concentrate on identifying duplicate vulnerabilities within vulnerability databases, relying primarily on textual descriptions and associated metadata.

8.3. Duplicate bug reports detection

Although our primary objective is to identify duplicate vulnerability records within vulnerability databases, this task closely resembles the established field of duplicate bug report detection. Both domains involve analyzing textual descriptions to determine whether separate records refer to the same underlying issue, making insights and methodologies from bug report detection highly relevant and instructive for our research. Duplicate bug report detection has been extensively studied for over a decade [21], resulting in a variety of methodologies. Initial approaches focused on leveraging surface features, textual semantics, and graph clustering to predict duplicate statuses [21]. Given that bug reports are predominantly textual, natural language processing techniques have been widely applied to enhance duplicate detection [22]. Additionally, information retrieval methods have been employed to measure similarity between bug reports. For instance, Sun et al. [23] introduced discriminative models for information retrieval to improve detection accuracy by developing a retrieval function that assesses both textual and non-textual fields such as product, component, and version. They further refined the retrieval function by extending the BM25F similarity formula and employing a two-round stochastic gradient descent process to optimize the retrieval function for specific bug repositories in a supervised manner [24,25]. Beyond these foundational techniques, researchers have integrated contextual software development information and domain-specific knowledge with information retrieval methods to better identify duplicate bug reports [55,56]. Additionally, Nguyen et al. [26] combined information retrieval with topic modeling, treating bug reports as documents describing technical issues and identifying duplicates based on shared technical themes. Learning-to-rank algorithms [57] have also been adopted to prioritize potential duplicates effectively. In recent years, deep learning models have become prominent in duplicate bug report detection, with architectures such as Convolutional Neural Networks (CNNs) [27] and dual-channel CNNs [28] being widely explored. However, the effectiveness of deep learning techniques in practical applications remains a topic of debate. Studies indicate that while deep learning models perform well when duplicate bug report detection is treated as a binary classification task, their superiority diminishes in ranking scenarios where the goal is to prioritize potential duplicates based on similarity rankings. Empirical evidence from [33] suggests that deep learning methods may underperform classical information retrieval approaches. The study also found that simpler similarity-based techniques often surpass more sophisticated models across various projects, aligning with our findings in duplicate vulnerability detection.

While both duplicate bug report detection and our focus on duplicate vulnerabilities aim to identify entries that describe the same underlying issue, key differences remain. First, duplicate bug report detection benefits from extensive research and large, well-established datasets (e.g., Mozilla, Bugzilla, Jira). These resources enable systematic evaluation and refinement of detection methods. In contrast, duplicate vulnerability detection lacks public datasets, prompting us to build and analyze one from scratch—a contribution of this study. Second, while bug reports often contain structured fields and contextual information, vulnerability data typically appears as unstructured text in various formats. Bug reports center on user-observed symptoms, often resulting in more textual similarity among duplicates. Vulnerability reports, however, focus on triggers, leading to less textual overlap. These differences demand adapting existing approaches to the variability of vulnerability data. By addressing these challenges, our work contributes to more efficient and reliable vulnerability management.

9. Conclusion

In this study, we tackled the significant challenge of detecting duplicate vulnerability records across multiple databases. To alleviate the manual workload for database maintainers and enhance vulnerability management, we developed a hybrid approach that combines cosine similarity with a fine-tuned BERT-based language model. Our pilot study and the creation of a comprehensive duplicate vulnerability dataset highlighted the diversity and complexity of vulnerability records across different databases. Experimental results demonstrated that our method consistently outperformed several baseline methods, achieving high accuracy while maintaining computational efficiency. The cosine similarity component enabled swift initial screening, while the fine-tuned BERT model captured deeper semantic relationships, thereby improving overall detection accuracy. This combination makes our method well-suited for

real-world database maintenance scenarios. Additionally, our method exhibited strong generalization capabilities when applied to new duplicate records, confirming its robustness and adaptability. Comparative analysis revealed that cosine similarity excelled with concise, structurally similar records, whereas BERT-based methods were superior for semantically complex cases. In summary, our hybrid method provides a practical and effective solution for identifying duplicate vulnerabilities, enhancing the accuracy and efficiency of vulnerability management processes, and laying the foundation for future advancements in automated duplicate detection.

CRediT authorship contribution statement

Kangliang Zhu: Validation, Methodology, Writing – original draft, Software, Data curation. **Wenhua Yang:** Writing – review & editing, Conceptualization, Writing – original draft, Validation. **Minxue Pan:** Validation, Writing – review & editing, Funding acquisition. **Yu Zhou:** Writing – review & editing, Funding acquisition, Validation.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

This work was partially supported by the National Natural Science Foundation of China (Nos. 62372232, 62372227), and the Collaborative Innovation Center of Novel Software Technology and Industrialization.

References

[1] S.A. Haryono, H.J. Kang, A. Sharma, A. Sharma, A. Santosa, A.M. Yi, D. Lo, Automated identification of libraries from vulnerability data: can we do better?, in: 2022 IEEE/ACM 30th International Conference on Program Comprehension (ICPC), 2022, pp. 178–189.
[2] Y. He, Y. Wang, S. Zhu, W. Wang, Y. Zhang, Q. Li, A. Yu, Automatically identifying cve affected versions with patches and developer logs, IEEE Trans. Dependable Secure Comput. 21 (2) (2024) 905–919, https://doi.org/10.1109/TDSC.2023.3264567.
[3] NVD, Cves and the nvd process, https://nvd.nist.gov/general/cve-process, 2025.
[4] OSV, Open source vulnerability, https://osv.dev/, 2024.
[5] A. Kimbrell, J. Kelly, Repair the bridge before it cracks: understanding vulnerabilities and weaknesses in modern it, https://www.redhat.com/zh/blog/repair-bridge-it-cracks-understanding-vulnerabilities-and-weaknesses-modern-it, 2025.
[6] C. Cifuentes, F. Gauthier, B. Hassanshahi, P. Krishnan, D. McCall, The role of program analysis in security vulnerability detection: then and now, Comput. Secur. 135 (C) (Dec. 2023), https://doi.org/10.1016/j.cose.2023.103463.
[7] Y. Chen, A.E. Santosa, A.M. Yi, A. Sharma, A. Sharma, D. Lo, A machine learning approach for vulnerability curation, in: Proceedings of the 17th International Conference on Mining Software Repositories, MSR '20, Association for Computing Machinery, New York, NY, USA, 2020, pp. 32–42.
[8] F. Wu, J. Wang, J. Liu, W. Wang, Vulnerability detection with deep learning, in: 2017 3rd IEEE International Conference on Computer and Communications (ICCC), 2017, pp. 1298–1302.
[9] X. Cheng, H. Wang, J. Hua, G. Xu, Y. Sui, Deepwukong: statically detecting software vulnerabilities using deep graph neural network, ACM Trans. Softw. Eng. Methodol. 30 (3) (Apr. 2021), https://doi.org/10.1145/3436877.
[10] D. Hin, A. Kan, H. Chen, M.A. Babar, Linevd: statement-level vulnerability detection using graph neural networks, in: Proceedings of the 19th International Conference on Mining Software Repositories, MSR '22, Association for Computing Machinery, New York, NY, USA, 2022, pp. 596–607.
[11] C. Thapa, S.I. Jang, M.E. Ahmed, S. Camtepe, J. Pieprzyk, S. Nepal, Transformer-based language models for software vulnerability detection, in: Proceedings of the 38th Annual Computer Security Applications Conference, ACSAC '22, Association for Computing Machinery, New York, NY, USA, 2022, pp. 481–496.
[12] N.H. Pham, T.T. Nguyen, H.A. Nguyen, T.N. Nguyen, Detection of recurring software vulnerabilities, in: Proceedings of the 25th IEEE/ACM International Conference on Automated Software Engineering, ASE '10, Association for Computing Machinery, New York, NY, USA, 2010, pp. 447–456.
[13] Z. Li, D. Zou, S. Xu, H. Jin, H. Qi, J. Hu, Vulpecker: an automated vulnerability detection system based on code similarity analysis, in: Proceedings of the 32nd Annual Conference on Computer Security Applications, ACSAC '16, Association for Computing Machinery, New York, NY, USA, 2016, pp. 201–213.
[14] Q. Zhao, C. Huang, L. Dai, Vuldeff: vulnerability detection method based on function fingerprints and code differences, Knowl.-Based Syst. 260 (2023) 110139, https://doi.org/10.1016/j.knosys.2022.110139.
[15] Z. Chen, S. Kommrusch, M. Monperrus, Neural transfer learning for repairing security vulnerabilities in c code, IEEE Trans. Softw. Eng. 49 (1) (2023) 147–165, https://doi.org/10.1109/TSE.2022.3147265.
[16] G. Bhandari, A. Naseer, L. Moonen, Cvefixes: automated collection of vulnerabilities and their fixes from open-source software, in: Proceedings of the 17th International Conference on Predictive Models and Data Analytics in Software Engineering, PROMISE 2021, Association for Computing Machinery, New York, NY, USA, 2021, pp. 30–39.
[17] A. Decan, T. Mens, E. Constantinou, On the impact of security vulnerabilities in the npm package dependency network, in: Proceedings of the 15th International Conference on Mining Software Repositories, MSR '18, Association for Computing Machinery, New York, NY, USA, 2018, pp. 181–191.
[18] S. Wu, R. Wang, K. Huang, Y. Cao, W. Song, Z. Zhou, Y. Huang, B. Chen, X. Peng, Vision: identifying affected library versions for open source software vulnerabilities, in: 2024 39th IEEE/ACM International Conference on Automated Software Engineering (ASE), IEEE Computer Society, Los Alamitos, CA, USA, 2024, pp. 1447–1459, https://doi.ieeecomputersociety.org/.
[19] S. Wu, W. Song, K. Huang, B. Chen, X. Peng, Identifying affected libraries and their ecosystems for open source software vulnerabilities, in: Proceedings of the IEEE/ACM 46th International Conference on Software Engineering, ICSE '24, Association for Computing Machinery, New York, NY, USA, 2024.
[20] W. Zheng, P. Deng, K. Gui, X. Wu, An abstract syntax tree based static fuzzing mutation for vulnerability evolution analysis, Inf. Softw. Technol. 158 (2023) 107194, https://doi.org/10.1016/j.infsof.2023.107194.
[21] N. Jalbert, W. Weimer, Automated duplicate detection for bug tracking systems, in: 2008 IEEE International Conference on Dependable Systems and Networks with FTCS and DCC (DSN), 2008, pp. 52–61.
[22] X. Wang, L. Zhang, T. Xie, J. Anvik, J. Sun, An approach to detecting duplicate bug reports using natural language and execution information, in: Proceedings of the 30th International Conference on Software Engineering, ICSE '08, Association for Computing Machinery, New York, NY, USA, 2008, pp. 461–470.
[23] C. Sun, D. Lo, X. Wang, J. Jiang, S.-C. Khoo, A discriminative model approach for accurate duplicate bug report retrieval, in: Proceedings of the 32nd ACM/IEEE International Conference on Software Engineering - Volume 1, ICSE '10, Association for Computing Machinery, New York, NY, USA, 2010, pp. 45–54.

[24] C. Sun, D. Lo, S.-C. Khoo, J. Jiang, Towards more accurate retrieval of duplicate bug reports, in: 2011 26th IEEE/ACM International Conference on Automated Software Engineering (ASE 2011), 2011, pp. 253–262.
[25] Y. Tian, C. Sun, D. Lo, Improved duplicate bug report identification, in: 2012 16th European Conference on Software Maintenance and Reengineering, 2012, pp. 385–390.
[26] A.T. Nguyen, T.T. Nguyen, T.N. Nguyen, D. Lo, C. Sun, Duplicate bug report detection with a combination of information retrieval and topic modeling, in: Proceedings of the 27th IEEE/ACM International Conference on Automated Software Engineering, ASE '12, Association for Computing Machinery, New York, NY, USA, 2012, pp. 70–79.
[27] Q. Xie, Z. Wen, J. Zhu, C. Gao, Z. Zheng, Detecting duplicate bug reports with convolutional neural networks, in: 2018 25th Asia-Pacific Software Engineering Conference (APSEC), 2018, pp. 416–425.
[28] J. He, L. Xu, M. Yan, X. Xia, Y. Lei, Duplicate bug report detection using dual-channel convolutional neural networks, in: Proceedings of the 28th International Conference on Program Comprehension, ICPC '20, Association for Computing Machinery, New York, NY, USA, 2020, pp. 117–127.
[29] J. Zhang, F. Wang, F. Ma, G. Song, Text similarity calculation method based on optimized cosine distance, in: 2022 International Conference on Electronics and Devices, Computational Science (ICEDCS), 2022, pp. 37–39.
[30] N.H. Sulaiman, D. Mohamad, A Jaccard-based similarity measure for soft sets, in: 2012 IEEE Symposium on Humanities, Science and Engineering Research, 2012, pp. 659–663.
[31] S. Zhu, J. Wu, G. Xia, Top-k cosine similarity interesting pairs search, in: 2010 Seventh International Conference on Fuzzy Systems and Knowledge Discovery, vol. 3, 2010, pp. 1479–1483.
[32] S. Jahan, M.M. Rahman, Towards understanding the impacts of textual dissimilarity on duplicate bug report detection, in: 2023 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER), 2023, pp. 25–36.
[33] T. Zhang, D. Han, V. Vinayakarao, I.C. Irsan, B. Xu, F. Thung, D. Lo, L. Jiang, Duplicate bug report detection: how far are we?, ACM Trans. Softw. Eng. Methodol. 32 (4) (May 2023), https://doi.org/10.1145/3576042.
[34] Y. Tian, D. Lo, C. Sun, Information retrieval based nearest neighbor classification for fine-grained bug severity prediction, in: 2012 19th Working Conference on Reverse Engineering, 2012, pp. 215–224.
[35] Y. Jiang, X. Su, C. Treude, C. Shang, T. Wang, Does deep learning improve the performance of duplicate bug report detection? An empirical study, J. Syst. Softw. 198 (2023) 111607, https://doi.org/10.1016/j.jss.2023.111607.
[36] C. Raffel, N.M. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, P.J. Liu, Exploring the limits of transfer learning with a unified text-to-text transformer, J. Mach. Learn. Res. 21 (2019) 140:1–140:67, https://api.semanticscholar.org/CorpusID:204838007.
[37] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, V. Stoyanov, Roberta: a robustly optimized bert pretraining approach, arXiv, arXiv:1907.11692 [abs], 2019, https://api.semanticscholar.org/CorpusID:198953378.
[38] P. He, X. Liu, J. Gao, W. Chen, Deberta: decoding-enhanced bert with disentangled attention, arXiv, arXiv:2006.03654 [abs], 2020, https://api.semanticscholar.org/CorpusID:219531210.
[39] K. Clark, Electra: pre-training text encoders as discriminators rather than generators, arXiv preprint, arXiv:2003.10555, 2020.
[40] Y. Xiao, B. Chen, C. Yu, Z. Xu, Z. Yuan, F. Li, B. Liu, Y. Liu, W. Huo, W. Zou, W. Shi, MVP: detecting vulnerabilities using patch-enhanced vulnerability signatures, in: 29th USENIX Security Symposium (USENIX Security 20), USENIX Association, 2020, pp. 1165–1182, https://www.usenix.org/conference/usenixsecurity20/presentation/xiao.
[41] W. Kang, B. Son, K. Heo, Tracer: signature-based static analysis for detecting recurring vulnerabilities, in: Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security, CCS '22, Association for Computing Machinery, New York, NY, USA, 2022, pp. 1695–1708.
[42] S. Feng, Y. Wu, W. Xue, S. Pan, D. Zou, Y. Liu, H. Jin, FIRE: combining multi-stage filtering with taint analysis for scalable recurring vulnerability detection, in: 33rd USENIX Security Symposium (USENIX Security 24), USENIX Association, Philadelphia, PA, 2024, pp. 1867–1884, https://www.usenix.org/conference/usenixsecurity24/presentation/feng-siyue.
[43] Y. Xiao, Z. Xu, W. Zhang, C. Yu, L. Liu, W. Zou, Z. Yuan, Y. Liu, A. Piao, W. Huo, Viva: binary level vulnerability identification via partial signature, in: 2021 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER), 2021, pp. 213–224.
[44] H. Xiao, Y. Zhang, M. Shen, C. Lin, C. Zhang, S. Liu, M. Yang, Accurate and efficient recurring vulnerability detection for iot firmware, in: Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security, CCS '24, Association for Computing Machinery, New York, NY, USA, 2024, pp. 3317–3331.
[45] Y. Shi, Y. Zhang, T. Bai, L. Zhang, X. Tan, M. Yang, Recurscan: detecting recurring vulnerabilities in php web applications, in: Proceedings of the ACM Web Conference 2024, WWW '24, Association for Computing Machinery, New York, NY, USA, 2024, pp. 1746–1755.
[46] L. Wang, Q. Zhang, P. Zhao, Automated detection of code vulnerabilities based on program analysis and model checking, in: 2008 Eighth IEEE International Working Conference on Source Code Analysis and Manipulation, 2008, pp. 165–173.
[47] J. Jang, A. Agrawal, D. Brumley, Redebug: finding unpatched code clones in entire os distributions, in: Proceedings of the 2012 IEEE Symposium on Security and Privacy, SP '12, IEEE Computer Society, USA, 2012, pp. 48–62.
[48] S. Kim, S. Woo, H. Lee, H. Oh, Vuddy: a scalable approach for vulnerable code clone discovery, in: 2017 IEEE Symposium on Security and Privacy (SP), 2017, pp. 595–614.
[49] N. Shiri Harzevili, A. Boaye Belle, J. Wang, S. Wang, Z.M.J. Jiang, N. Nagappan, A systematic literature review on automated software vulnerability detection using machine learning, ACM Comput. Surv. 57 (3) (Nov. 2024), https://doi.org/10.1145/3699711.
[50] G. Lin, S. Wen, Q.-L. Han, J. Zhang, Y. Xiang, Software vulnerability detection using deep neural networks: a survey, Proc. IEEE 108 (10) (2020) 1825–1848, https://doi.org/10.1109/JPROC.2020.2993293.
[51] B. Steenhoek, M.M. Rahman, R. Jiles, W. Le, An empirical study of deep learning models for vulnerability detection, in: 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE), 2023, pp. 2237–2248.
[52] S. Chakraborty, R. Krishna, Y. Ding, B. Ray, Deep learning based vulnerability detection: are we there yet?, IEEE Trans. Softw. Eng. 48 (9) (2022) 3280–3296, https://doi.org/10.1109/TSE.2021.3087402.
[53] X. Yu, G. Lin, X. Hu, J.W. Keung, X. Xia, Less is more: unlocking semi-supervised deep learning for vulnerability detection, ACM Trans. Softw. Eng. Methodol. 34 (3) (2025) 62, https://doi.org/10.1145/3699602.
[54] G. Lu, X. Ju, X. Chen, W. Pei, Z. Cai, Grace: empowering llm-based software vulnerability detection with graph structure and in-context learning, J. Syst. Softw. 212 (C) (Jul. 2024), https://doi.org/10.1016/j.jss.2024.112031.
[55] A. Alipour, A. Hindle, E. Stroulia, A contextual approach towards more accurate duplicate bug report detection, in: 2013 10th Working Conference on Mining Software Repositories (MSR), 2013, pp. 183–192.
[56] K. Aggarwal, F. Timbers, T. Rutgers, A. Hindle, E. Stroulia, R. Greiner, Detecting duplicate bug reports with software engineering domain knowledge, J. Softw. Evol. Process 29 (3) (2017) e1821, https://doi.org/10.1002/smr.1821.
[57] J. Zhou, H. Zhang, Learning to rank duplicate bug reports, in: Proceedings of the 21st ACM International Conference on Information and Knowledge Management, CIKM '12, Association for Computing Machinery, New York, NY, USA, 2012, pp. 852–861.